Java Programming

(PAMCA502) [Assessment- IV]

Aim

To develop a JavaFX-based desktop application that allows users to securely manage file metadata through a GUI interface, validate input data, and store/retrieve records from a MySQL database using JDBC (via Spring Boot and Hibernate ORM).

Procedure

Database Setup

- Create a MySQL database named securevault.
- The required tables for storing file details and audit logs are automatically created by the application.

JDBC Configuration

- Enter the database URL, username, password, and MySQL driver in the configuration settings.
- The MySQL Connector/J driver establishes the JDBC connection between the application and the database.

Data Model

- Define a class to represent one record in the database table.
- Each object corresponds to one row storing file metadata.

Database Operations

- Implement a layer to perform database operations such as inserting and retrieving records.
- The application stores and fetches file information from the database.

Business Logic

- Add processing logic before saving data.
- The system analyzes file information and assigns a risk level.

Server Handling

- The server receives requests from the client application.
- It processes the request and sends a response back.

GUI Design

- Create the user interface using JavaFX controls like text fields, buttons, labels, and tables.
- The user can log in, add file details, and view stored records.

Client–Server Communication

- The client application sends HTTP requests to the server and receives responses in JSON format.

Records Display

- Retrieve stored data from the server and display it in a table on the screen.

Application Execution

- Start the backend server first.
- Then launch the GUI application and interact with the system.

Verification

- Insert a record through the GUI and verify that it appears in the MySQL database.

CLIENT (JavaFX GUI)

Entry (program starts here):

MainApp.java

```
package com.securevaultclient;

import com.securevaultclient.util.SceneSwitcher;
import javafx.application.Application;
import javafx.stage.Stage;

public class MainApp extends Application {

    private static MainApp instance;
    private Stage primaryStage;
    private String currentUsername;

    @Override
    public void start(Stage stage) {
        instance = this;
        this.primaryStage = stage;
        this.primaryStage.setTitle("Secure Digital Vault");
        this.primaryStage.setResizable(false);

        SceneSwitcher.switchTo("login");
    }

    public static MainApp getInstance() {
        return instance;
```

```
    }

    public Stage getPrimaryStage() {
        return primaryStage;
    }

    public String getCurrentUsername() {
        return currentUsername;
    }

    public void setCurrentUsername(String username) {
        this.currentUsername = username;
    }

    public static void main(String[] args) {
        launch(args);
    }
}
```

MainApp.java × | login.fxml × | dashboard.fxml × | view_files.fxml × | LoginController.java

Source History

```java
package com.securevaultclient;

import com.securevaultclient.util.SceneSwitcher;
import javafx.application.Application;
import javafx.stage.Stage;

public class MainApp extends Application {

    private static MainApp instance;
    private Stage primaryStage;
    private String currentUsername;

    @Override
    public void start(Stage stage) {
        instance = this;
        this.primaryStage = stage;
        this.primaryStage.setTitle("Secure Digital Vault");
        this.primaryStage.setResizable(false);

        SceneSwitcher.switchTo("login");
    }

    public static MainApp getInstance() {
        return instance;
    }

    public Stage getPrimaryStage() {
        return primaryStage;
    }

    public String getCurrentUsername() {
        return currentUsername;
    }

    public void setCurrentUsername(String username) {
        this.currentUsername = username;
    }

    public static void main(String[] args) {
        launch(args);
    }
}
```

GUI Screens (JavaFx)

login.fxml

```
<?xml version="1.0" encoding="UTF-8"?>


<?import javafx.geometry.Insets?>

<?import javafx.scene.control.Button?>

<?import javafx.scene.control.Label?>

<?import javafx.scene.control.TextField?>

<?import javafx.scene.layout.VBox?>

<?import javafx.scene.text.Font?>


<VBox alignment="CENTER" spacing="20" xmlns="http://javafx.com/javafx/17"
xmlns:fx="http://javafx.com/fxml/1"
fx:controller="com.securevaultclient.controller.LoginController">

   <padding>

      <Insets bottom="30" left="30" right="30" top="30" />

   </padding>


   <Label text="Secure Digital Vault" textFill="#2c3e50">

      <font>

         <Font name="System Bold" size="24" />

      </font>

   </Label>


   <Label text="Login" textFill="#34495e">

      <font>

         <Font size="18" />

      </font>
```

```
    </Label>

    <TextField fx:id="usernameField" promptText="Enter username" maxWidth="250" />

    <Button fx:id="loginButton" defaultButton="true" maxWidth="250"
mnemonicParsing="false" onAction="#handleLogin" style="-fx-background-color: #3498db; -
fx-text-fill: white;" text="Login">
        <font>
            <Font size="14" />
        </font>
    </Button>
</VBox>
```

MainApp.java × | login.fxml × | dashboard.fxml × | view_files.fxml × | LoginController.java × | Dashb

```xml
<?xml version="1.0" encoding="UTF-8"?>

<?import javafx.geometry.Insets?>
<?import javafx.scene.control.Button?>
<?import javafx.scene.control.Label?>
<?import javafx.scene.control.TextField?>
<?import javafx.scene.layout.VBox?>
<?import javafx.scene.text.Font?>

<VBox alignment="CENTER" spacing="20" xmlns="http://javafx.com/javafx/17" xmlns:
    <padding>
        <Insets bottom="30" left="30" right="30" top="30" />
    </padding>

    <Label text="Secure Digital Vault" textFill="#2c3e50">
        <font>
            <Font name="System Bold" size="24" />
        </font>
    </Label>

    <Label text="Login" textFill="#34495e">
        <font>
            <Font size="18" />
        </font>
    </Label>

    <TextField fx:id="usernameField" promptText="Enter username" maxWidth="250"

    <Button fx:id="loginButton" defaultButton="true" maxWidth="250" mnemonicPars
        <font>
            <Font size="14" />
        </font>
    </Button>
</VBox>
```

dashboard.fxml

<?xml version="1.0" encoding="UTF-8"?>

<?import javafx.geometry.Insets?>

```
<?import javafx.scene.control.Button?>
<?import javafx.scene.control.Label?>
<?import javafx.scene.layout.BorderPane?>
<?import javafx.scene.layout.GridPane?>
<?import javafx.scene.layout.VBox?>
<?import javafx.scene.text.Font?>

<BorderPane xmlns="http://javafx.com/javafx/17" xmlns:fx="http://javafx.com/fxml/1"
fx:controller="com.securevaultclient.controller.DashboardController">

    <top>
        <VBox alignment="CENTER" spacing="10" style="-fx-background-color: #2c3e50;">
            <padding>
                <Insets bottom="15" left="15" right="15" top="15" />
            </padding>

            <Label fx:id="welcomeLabel" text="Welcome!" textFill="white">
                <font>
                    <Font name="System Bold" size="20" />
                </font>
            </Label>
        </VBox>
    </top>

    <center>
        <GridPane alignment="CENTER" hgap="20" vgap="20">
            <padding>
```

```
            <Insets bottom="30" left="30" right="30" top="30" />
        </padding>

        <Button fx:id="addFileBtn" mnemonicParsing="false" onAction="#handleAddFile"
prefHeight="100" prefWidth="150" style="-fx-background-color: #27ae60; -fx-text-fill: white;"
text="Add File">
            <font>
                <Font size="14" />
            </font>
        </Button>

        <Button fx:id="viewFilesBtn" mnemonicParsing="false" onAction="#handleViewFiles"
prefHeight="100" prefWidth="150" style="-fx-background-color: #2980b9; -fx-text-fill: white;"
text="View Files" GridPane.columnIndex="1">
            <font>
                <Font size="14" />
            </font>
        </Button>

        <Button fx:id="verifyBtn" mnemonicParsing="false" onAction="#handleVerifyFile"
prefHeight="100" prefWidth="150" style="-fx-background-color: #f39c12; -fx-text-fill: white;"
text="Verify File" GridPane.rowIndex="1">
            <font>
                <Font size="14" />
            </font>
        </Button>

        <Button fx:id="auditBtn" mnemonicParsing="false" onAction="#handleViewAudit"
prefHeight="100" prefWidth="150" style="-fx-background-color: #8e44ad; -fx-text-fill: white;"
text="View Audit" GridPane.columnIndex="1" GridPane.rowIndex="1">
```

```
            <font>
                <Font size="14" />
            </font>
        </Button>


        <Button fx:id="logoutBtn" mnemonicParsing="false" onAction="#handleLogout"
prefHeight="50" prefWidth="320" style="-fx-background-color: #e74c3c; -fx-text-fill: white;"
text="Logout" GridPane.columnSpan="2" GridPane.rowIndex="2">
            <font>
                <Font size="14" />
            </font>
        </Button>
    </GridPane>
  </center>
</BorderPane>
```

MainApp.java × | login.fxml × | dashboard.fxml × | view_files.fxml × | LoginController.java × | Dashboa

```
14          <VBox alignment="CENTER" spacing="10" style="-fx-background-color: #2c3e50
15              <padding>
16                  <Insets bottom="15" left="15" right="15" top="15" />
17              </padding>
18
19              <Label fx:id="welcomeLabel" text="Welcome!" textFill="white">
20                  <font>
21                      <Font name="System Bold" size="20" />
22                  </font>
23              </Label>
24          </VBox>
25      </top>
26
27      <center>
28          <GridPane alignment="CENTER" hgap="20" vgap="20">
29              <padding>
30                  <Insets bottom="30" left="30" right="30" top="30" />
31              </padding>
32
33              <Button fx:id="addFileBtn" mnemonicParsing="false" onAction="#handleAd
34                  <font>
35                      <Font size="14" />
36                  </font>
37              </Button>
38
39              <Button fx:id="viewFilesBtn" mnemonicParsing="false" onAction="#handle
40                  <font>
41                      <Font size="14" />
42                  </font>
43              </Button>
44
45              <Button fx:id="verifyBtn" mnemonicParsing="false" onAction="#handleVer
46                  <font>
47                      <Font size="14" />
48                  </font>
49              </Button>
50
51              <Button fx:id="auditBtn" mnemonicParsing="false" onAction="#handleView
52                  <font>
53                      <Font size="14" />
54                  </font>
55              </Button>
56
57              <Button fx:id="logoutBtn" mnemonicParsing="false" onAction="#handleLog
58                  <font>
59                      <Font size="14" />
60                  </font>
```

view_files.fxml

```
<?xml version="1.0" encoding="UTF-8"?>
```

```
<?import javafx.geometry.Insets?>
<?import javafx.scene.control.Button?>
<?import javafx.scene.control.Label?>
<?import javafx.scene.control.TableColumn?>
<?import javafx.scene.control.TableView?>
<?import javafx.scene.layout.BorderPane?>
<?import javafx.scene.layout.HBox?>
<?import javafx.scene.layout.VBox?>
<?import javafx.scene.text.Font?>

<BorderPane xmlns="http://javafx.com/javafx/17" xmlns:fx="http://javafx.com/fxml/1"
fx:controller="com.securevaultclient.controller.ViewFilesController">

    <top>
        <VBox alignment="CENTER" spacing="10" style="-fx-background-color: #2980b9;">
            <padding>
                <Insets bottom="15" left="15" right="15" top="15" />
            </padding>

            <Label text="View Files" textFill="white">
                <font>
                    <Font name="System Bold" size="20" />
                </font>
            </Label>
        </VBox>
    </top>
```

```
<center>
  <VBox spacing="10">
    <padding>
      <Insets bottom="15" left="15" right="15" top="15" />
    </padding>

    <TableView fx:id="filesTable" prefHeight="400">
      <columns>
        <TableColumn fx:id="idColumn" prefWidth="50" text="ID" />
        <TableColumn fx:id="fileNameColumn" prefWidth="200" text="File Name" />
        <TableColumn fx:id="ownerColumn" prefWidth="100" text="Owner" />
        <TableColumn fx:id="fileTypeColumn" prefWidth="80" text="Type" />
        <TableColumn fx:id="sizeColumn" prefWidth="100" text="Size (bytes)" />
        <TableColumn fx:id="uploadDateColumn" prefWidth="150" text="Upload Date" />
        <TableColumn fx:id="riskLevelColumn" prefWidth="80" text="Risk" />
      </columns>
    </TableView>

    <HBox alignment="CENTER" spacing="20">
      <Button fx:id="refreshBtn" mnemonicParsing="false" onAction="#handleRefresh"
prefWidth="120" style="-fx-background-color: #3498db; -fx-text-fill: white;" text="Refresh">
        <font>
          <Font size="14" />
        </font>
      </Button>
```

```
            <Button fx:id="backBtn" mnemonicParsing="false" onAction="#handleBack"
prefWidth="120" style="-fx-background-color: #95a5a6; -fx-text-fill: white;" text="Back">
                <font>
                    <Font size="14" />
                </font>
            </Button>
        </HBox>
    </VBox>
  </center>
</BorderPane>
```

MainApp.java × | login.fxml × | dashboard.fxml × | view_files.fxml × | LoginController.java × | DashboardController.java

```
<?import javafx.scene.layout.VBox?>
<?import javafx.scene.text.Font?>

<BorderPane xmlns="http://javafx.com/javafx/17" xmlns:fx="http://javafx.com/fxml/1" fx:contro

    <top>
        <VBox alignment="CENTER" spacing="10" style="-fx-background-color: #2980b9;">
            <padding>
                <Insets bottom="15" left="15" right="15" top="15" />
            </padding>

            <Label text="View Files" textFill="white">
                <font>
                    <Font name="System Bold" size="20" />
                </font>
            </Label>
        </VBox>
    </top>

    <center>
        <VBox spacing="10">
            <padding>
                <Insets bottom="15" left="15" right="15" top="15" />
            </padding>

            <TableView fx:id="filesTable" prefHeight="400">
                <columns>
                    <TableColumn fx:id="idColumn" prefWidth="50" text="ID" />
                    <TableColumn fx:id="fileNameColumn" prefWidth="200" text="File Name" />
                    <TableColumn fx:id="ownerColumn" prefWidth="100" text="Owner" />
                    <TableColumn fx:id="fileTypeColumn" prefWidth="80" text="Type" />
                    <TableColumn fx:id="sizeColumn" prefWidth="100" text="Size (bytes)" />
                    <TableColumn fx:id="uploadDateColumn" prefWidth="150" text="Upload Date"
                    <TableColumn fx:id="riskLevelColumn" prefWidth="80" text="Risk" />
                </columns>
            </TableView>

            <HBox alignment="CENTER" spacing="20">
                <Button fx:id="refreshBtn" mnemonicParsing="false" onAction="#handleRefresh"
                    <font>
                        <Font size="14" />
                    </font>
                </Button>

                <Button fx:id="backBtn" mnemonicParsing="false" onAction="#handleBack" prefWi
                    <font>
                        <Font size="14" />
                    </font>
                </Button>
            </HBox>
```

Controllers (button actions / event handling)

LoginController.java

```
package com.securevaultclient.controller;

import com.securevaultclient.MainApp;
import com.securevaultclient.service.ApiService;
import com.securevaultclient.util.AlertUtil;
import com.securevaultclient.util.SceneSwitcher;
import javafx.fxml.FXML;
import javafx.scene.control.TextField;
import javafx.scene.input.KeyCode;

public class LoginController {

    @FXML
    private TextField usernameField;

    @FXML
    public void initialize() {
        usernameField.setOnKeyPressed(event -> {
            if (event.getCode() == KeyCode.ENTER) {
                handleLogin();
            }
        });
    }
```

```
    @FXML
    private void handleLogin() {
        String username = usernameField.getText().trim();

        if (username.isEmpty()) {
            AlertUtil.showWarning("Login Error", "Please enter a username");
            return;
        }

        // Store username in MainApp and ApiService
        MainApp.getInstance().setCurrentUsername(username);
        ApiService.getInstance().setCurrentUsername(username);

        AlertUtil.showInfo("Login Successful", "Welcome, " + username + "!");
        SceneSwitcher.switchTo("dashboard");
    }
}
```

MainApp.java × login.fxml × dashboard.fxml × view_files.fxml × LoginController.java × Dashboa

Source History

```java
package com.securevaultclient.controller;

import com.securevaultclient.MainApp;
import com.securevaultclient.service.ApiService;
import com.securevaultclient.util.AlertUtil;
import com.securevaultclient.util.SceneSwitcher;
import javafx.fxml.FXML;
import javafx.scene.control.TextField;
import javafx.scene.input.KeyCode;

public class LoginController {

    @FXML
    private TextField usernameField;

    @FXML
    public void initialize() {
        usernameField.setOnKeyPressed(event -> {
            if (event.getCode() == KeyCode.ENTER) {
                handleLogin();
            }
        });
    }

    @FXML
    private void handleLogin() {
        String username = usernameField.getText().trim();

        if (username.isEmpty()) {
            AlertUtil.showWarning("Login Error", "Please enter a username");
            return;
        }

        // Store username in MainApp and ApiService
        MainApp.getInstance().setCurrentUsername(username);
        ApiService.getInstance().setCurrentUsername(username);

        AlertUtil.showInfo("Login Successful", "Welcome, " + username + "!");
        SceneSwitcher.switchTo("dashboard");
    }
```

DashboardController.java

package com.securevaultclient.controller;

import com.securevaultclient.MainApp;

import com.securevaultclient.service.ApiService;

import com.securevaultclient.util.AlertUtil;

import com.securevaultclient.util.SceneSwitcher;

import javafx.fxml.FXML;

import javafx.scene.control.Label;

```
public class DashboardController {

    @FXML
    private Label welcomeLabel;

    @FXML
    public void initialize() {
        String username = MainApp.getInstance().getCurrentUsername();
        welcomeLabel.setText("Welcome, " + username + "!");
    }

    @FXML
    private void handleAddFile() {
        SceneSwitcher.switchTo("add_file");
    }

    @FXML
    private void handleViewFiles() {
        SceneSwitcher.switchTo("view_files");
    }

    @FXML
    private void handleVerifyFile() {
        SceneSwitcher.switchTo("verify");
    }
```

```
    @FXML
    private void handleViewAudit() {
        SceneSwitcher.switchTo("audit");
    }

    @FXML
    private void handleLogout() {
        if (AlertUtil.showConfirmation("Logout", "Are you sure you want to logout?")) {
            MainApp.getInstance().setCurrentUsername(null);
            ApiService.getInstance().setCurrentUsername(null);
            SceneSwitcher.switchTo("login");
        }
    }
}
```

MainApp.java × login.fxml × dashboard.fxml × view_files.fxml × LoginController.java × DashboardController.jav

Source History

```java
package com.securevaultclient.controller;

import com.securevaultclient.MainApp;
import com.securevaultclient.service.ApiService;
import com.securevaultclient.util.AlertUtil;
import com.securevaultclient.util.SceneSwitcher;
import javafx.fxml.FXML;
import javafx.scene.control.Label;

public class DashboardController {

    @FXML
    private Label welcomeLabel;

    @FXML
    public void initialize() {
        String username = MainApp.getInstance().getCurrentUsername();
        welcomeLabel.setText("Welcome, " + username + "!");
    }

    @FXML
    private void handleAddFile() {
        SceneSwitcher.switchTo("add_file");
    }

    @FXML
    private void handleViewFiles() {
        SceneSwitcher.switchTo("view_files");
    }

    @FXML
    private void handleVerifyFile() {
        SceneSwitcher.switchTo("verify");
    }

    @FXML
    private void handleViewAudit() {
        SceneSwitcher.switchTo("audit");
    }

    @FXML
    private void handleLogout() {
        if (AlertUtil.showConfirmation("Logout", "Are you sure you want to logout?")) {
            MainApp.getInstance().setCurrentUsername(null);
            ApiService.getInstance().setCurrentUsername(null);
            SceneSwitcher.switchTo("login");
        }
    }
}
```

ViewFilesController.java

package com.securevaultclient.controller;

```
import com.securevaultclient.model.FileMetadataModel;
import com.securevaultclient.service.ApiService;
import com.securevaultclient.util.AlertUtil;
import com.securevaultclient.util.SceneSwitcher;
import javafx.collections.FXCollections;
import javafx.collections.ObservableList;
import javafx.fxml.FXML;
import javafx.scene.control.TableColumn;
import javafx.scene.control.TableView;
import javafx.scene.control.cell.PropertyValueFactory;

public class ViewFilesController {

    @FXML
    private TableView<FileMetadataModel> filesTable;

    @FXML
    private TableColumn<FileMetadataModel, String> fileNameColumn;

    @FXML
    private TableColumn<FileMetadataModel, String> ownerColumn;

    @FXML
    private TableColumn<FileMetadataModel, String> fileTypeColumn;

    @FXML
    private TableColumn<FileMetadataModel, String> riskLevelColumn;
```

```
    private final ObservableList<FileMetadataModel> filesList =
FXCollections.observableArrayList();

    @FXML
    public void initialize() {

        // connect table columns with model properties
        fileNameColumn.setCellValueFactory(new PropertyValueFactory<>("fileName"));
        ownerColumn.setCellValueFactory(new PropertyValueFactory<>("owner"));
        fileTypeColumn.setCellValueFactory(new PropertyValueFactory<>("fileType"));
        riskLevelColumn.setCellValueFactory(new PropertyValueFactory<>("riskLevel"));

        loadFiles();
    }

    private void loadFiles() {
        try {
            filesList.clear();
            filesList.addAll(ApiService.getInstance().getAllFiles());
            filesTable.setItems(filesList);
        } catch (Exception e) {
            AlertUtil.showError("Error", "Failed to load files: " + e.getMessage());
        }
    }

    @FXML
```

```
    private void handleRefresh() {
        loadFiles();
    }

    @FXML
    private void handleBack() {
        SceneSwitcher.switchTo("dashboard");
    }
}
```

MainApp.java × login.fxml × dashboard.fxml × view_files.fxml × LoginController.java × DashboardController.java

Source History

```java
import javafx.scene.control.TableColumn;
import javafx.scene.control.TableView;
import javafx.scene.control.cell.PropertyValueFactory;

public class ViewFilesController {

    @FXML
    private TableView<FileMetadataModel> filesTable;

    @FXML
    private TableColumn<FileMetadataModel, String> fileNameColumn;

    @FXML
    private TableColumn<FileMetadataModel, String> ownerColumn;

    @FXML
    private TableColumn<FileMetadataModel, String> fileTypeColumn;

    @FXML
    private TableColumn<FileMetadataModel, String> riskLevelColumn;

    private final ObservableList<FileMetadataModel> filesList = FXCollections.observableArrayI

    @FXML
    public void initialize() {

        // connect table columns with model properties
        fileNameColumn.setCellValueFactory(new PropertyValueFactory<>("fileName"));
        ownerColumn.setCellValueFactory(new PropertyValueFactory<>("owner"));
        fileTypeColumn.setCellValueFactory(new PropertyValueFactory<>("fileType"));
        riskLevelColumn.setCellValueFactory(new PropertyValueFactory<>("riskLevel"));

        loadFiles();
    }

    private void loadFiles() {
        try {
            filesList.clear();
            filesList.addAll(ApiService.getInstance().getAllFiles());
            filesTable.setItems(filesList);
        } catch (Exception e) {
            AlertUtil.showError("Error", "Failed to load files: " + e.getMessage());
        }
    }

```

Integration

ApiService.java

package com.securevaultclient.service;

import com.fasterxml.jackson.core.type.TypeReference;

```
import com.fasterxml.jackson.databind.DeserializationFeature;
import com.fasterxml.jackson.databind.ObjectMapper;
import com.fasterxml.jackson.datatype.jsr310.JavaTimeModule;
import com.securevaultclient.model.AuditLogModel;
import com.securevaultclient.model.FileMetadataModel;
import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;
import java.util.List;

public class ApiService {

    private static final String BASE_URL = "http://localhost:8081/api";
    private static ApiService instance;
    private final HttpClient httpClient;
    private final ObjectMapper objectMapper;
    private String currentUsername;

    private ApiService() {
        this.httpClient = HttpClient.newHttpClient();
        this.objectMapper = new ObjectMapper();
        this.objectMapper.registerModule(new JavaTimeModule());

this.objectMapper.configure(DeserializationFeature.FAIL_ON_UNKNOWN_PROPERTIES,
false);
    }
```

```
public static ApiService getInstance() {
    if (instance == null) {
        instance = new ApiService();
    }
    return instance;
}

public void setCurrentUsername(String username) {
    this.currentUsername = username;
}

public FileMetadataModel addFile(String fileName, String fileType, Long size, String owner)
        throws Exception {
    String json = objectMapper.writeValueAsString(
        new FileMetadataModel(null, fileName, owner, fileType, size, null, null, null)
    );

    HttpRequest request = HttpRequest.newBuilder()
        .uri(URI.create(BASE_URL + "/files/add"))
        .header("Content-Type", "application/json")
        .header("username", currentUsername)
        .POST(HttpRequest.BodyPublishers.ofString(json))
        .build();

    HttpResponse<String> response = httpClient.send(request,
        HttpResponse.BodyHandlers.ofString());
```

```
    if (response.statusCode() == 200) {
        return objectMapper.readValue(response.body(), FileMetadataModel.class);
    } else {
        throw new RuntimeException("Error adding file: " + response.body());
    }
}

public List<FileMetadataModel> getAllFiles() throws Exception {
    HttpRequest request = HttpRequest.newBuilder()
        .uri(URI.create(BASE_URL + "/files/all"))
        .header("username", currentUsername)
        .GET()
        .build();

    HttpResponse<String> response = httpClient.send(request,
        HttpResponse.BodyHandlers.ofString());

    if (response.statusCode() == 200) {
        return objectMapper.readValue(response.body(),
            new TypeReference<List<FileMetadataModel>>() {});
    } else {
        throw new RuntimeException("Error getting files: " + response.body());
    }
}

public String verifyFile(Long fileId) throws Exception {
    HttpRequest request = HttpRequest.newBuilder()
```

```
        .uri(URI.create(BASE_URL + "/files/verify/" + fileId))
        .header("Content-Type", "application/json")
        .header("username", currentUsername)
        .POST(HttpRequest.BodyPublishers.noBody())
        .build();

    HttpResponse<String> response = httpClient.send(request,
        HttpResponse.BodyHandlers.ofString());

    if (response.statusCode() == 200) {
        var jsonNode = objectMapper.readTree(response.body());
        return jsonNode.get("status").asText() + ": " + jsonNode.get("message").asText();
    } else {
        throw new RuntimeException("Error verifying file: " + response.body());
    }
}

public List<AuditLogModel> getAllAuditLogs() throws Exception {
    HttpRequest request = HttpRequest.newBuilder()
        .uri(URI.create(BASE_URL + "/audit/all"))
        .header("username", currentUsername)
        .GET()
        .build();

    HttpResponse<String> response = httpClient.send(request,
        HttpResponse.BodyHandlers.ofString());
```

```
        if (response.statusCode() == 200) {
            return objectMapper.readValue(response.body(),
                new TypeReference<List<AuditLogModel>>() {});
        } else {
            throw new RuntimeException("Error getting audit logs: " + response.body());
        }
    }
}
```

MainApp.java × login.fxml × dashboard.fxml × view_files.fxml × LoginController.java × DashboardController.java ×

Source History

```java
        this.httpClient = HttpClient.newHttpClient();
        this.objectMapper = new ObjectMapper();
        this.objectMapper.registerModule(new JavaTimeModule());
        this.objectMapper.configure(DeserializationFeature.FAIL_ON_UNKNOWN_PROPERTIES, false);
    }

    public static ApiService getInstance() {
        if (instance == null) {
            instance = new ApiService();
        }
        return instance;
    }

    public void setCurrentUsername(String username) {
        this.currentUsername = username;
    }

    public FileMetadataModel addFile(String fileName, String fileType, Long size, String owner)
            throws Exception {
        String json = objectMapper.writeValueAsString(
            new FileMetadataModel(null, fileName, owner, fileType, size, null, null, null)
        );

        HttpRequest request = HttpRequest.newBuilder()
            .uri(URI.create(BASE_URL + "/files/add"))
            .header("Content-Type", "application/json")
            .header("username", currentUsername)
            .POST(HttpRequest.BodyPublishers.ofString(json))
            .build();

        HttpResponse<String> response = httpClient.send(request,
            HttpResponse.BodyHandlers.ofString());

        if (response.statusCode() == 200) {
            return objectMapper.readValue(response.body(), FileMetadataModel.class);
        } else {
            throw new RuntimeException("Error adding file: " + response.body());
        }
    }

    public List<FileMetadataModel> getAllFiles() throws Exception {
        HttpRequest request = HttpRequest.newBuilder()
            .uri(URI.create(BASE_URL + "/files/all"))
            .header("username", currentUsername)
            .GET()
```

Model (TableView data binding)

FileMetadataModel.java

package com.securevaultclient.model;

import com.fasterxml.jackson.annotation.JsonProperty;

```
import java.time.LocalDateTime;

public class FileMetadataModel {
    private Long id;

    @JsonProperty("fileName")
    private String fileName;

    private String owner;

    @JsonProperty("fileType")
    private String fileType;

    private Long size;
    private LocalDateTime uploadDate;
    private String hashValue;
    private String riskLevel;

    public FileMetadataModel() {}

    public FileMetadataModel(Long id, String fileName, String owner, String fileType,
                             Long size, LocalDateTime uploadDate, String hashValue, String riskLevel) {
        this.id = id;
        this.fileName = fileName;
        this.owner = owner;
        this.fileType = fileType;
        this.size = size;
```

```
        this.uploadDate = uploadDate;
        this.hashValue = hashValue;
        this.riskLevel = riskLevel;
    }

    // Getters and Setters
    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }

    public String getFileName() { return fileName; }
    public void setFileName(String fileName) { this.fileName = fileName; }

    public String getOwner() { return owner; }
    public void setOwner(String owner) { this.owner = owner; }

    public String getFileType() { return fileType; }
    public void setFileType(String fileType) { this.fileType = fileType; }

    public Long getSize() { return size; }
    public void setSize(Long size) { this.size = size; }

    public LocalDateTime getUploadDate() { return uploadDate; }
    public void setUploadDate(LocalDateTime uploadDate) { this.uploadDate = uploadDate; }

    public String getHashValue() { return hashValue; }
    public void setHashValue(String hashValue) { this.hashValue = hashValue; }
```

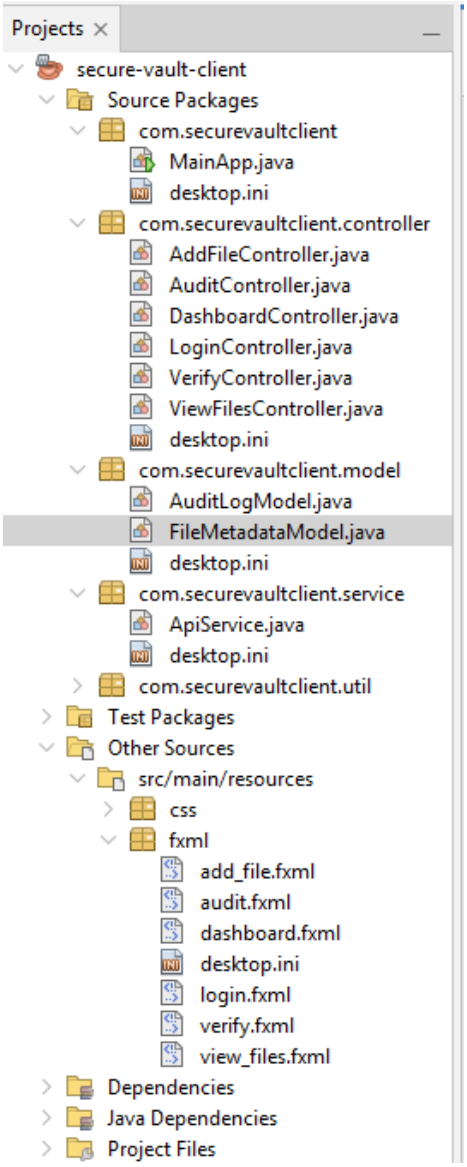
```
    public String getRiskLevel() { return riskLevel; }

    public void setRiskLevel(String riskLevel) { this.riskLevel = riskLevel; }

}
```

FileMetadataModel.java

Source History

```java
package com.securevaultclient.model;

import com.fasterxml.jackson.annotation.JsonProperty;
import java.time.LocalDateTime;

public class FileMetadataModel {
    private Long id;

    @JsonProperty("fileName")
    private String fileName;

    private String owner;

    @JsonProperty("fileType")
    private String fileType;

    private Long size;
    private LocalDateTime uploadDate;
    private String hashValue;
    private String riskLevel;

    public FileMetadataModel() {}

    public FileMetadataModel(Long id, String fileName, String owner, String fileType,
                             Long size, LocalDateTime uploadDate, String hashValue, Strin
        this.id = id;
        this.fileName = fileName;
        this.owner = owner;
        this.fileType = fileType;
        this.size = size;
        this.uploadDate = uploadDate;
        this.hashValue = hashValue;
        this.riskLevel = riskLevel;
    }

    // Getters and Setters
    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }

    public String getFileName() { return fileName; }
    public void setFileName(String fileName) { this.fileName = fileName; }

    public String getOwner() { return owner; }
    public void setOwner(String owner) { this.owner = owner; }
```

File Structure (Frontend):



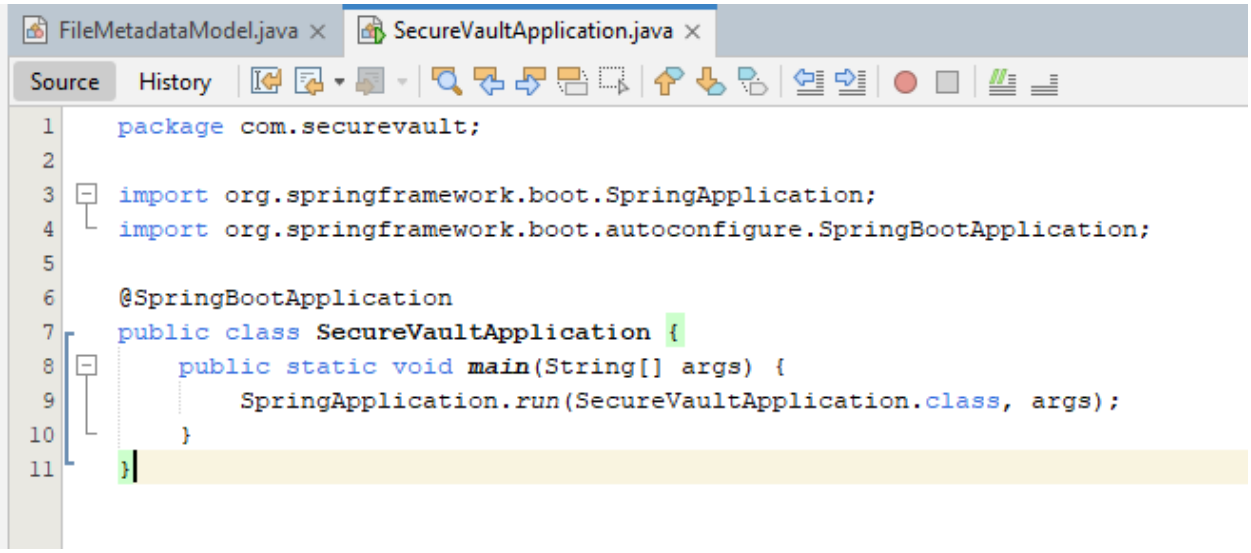
SERVER (Spring Boot Backend)

Server Entry Point

SecureVaultApplication.java

```
package com.securevault;


import org.springframework.boot.SpringApplication;

import org.springframework.boot.autoconfigure.SpringBootApplication;
```

```
@SpringBootApplication
public class SecureVaultApplication {
    public static void main(String[] args) {
        SpringApplication.run(SecureVaultApplication.class, args);
    }
}
```



REST API Layer (client communicates here)

VaultController.java

```
package com.securevault.controller;

import com.securevault.dto.FileRequestDTO;
import com.securevault.dto.FileResponseDTO;
import com.securevault.dto.VerifyResponseDTO;
import com.securevault.model.AuditLog;
import com.securevault.service.AuditService;
import com.securevault.service.VaultService;
```

```
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import java.util.List;

@RestController
@RequestMapping("/api")
@CrossOrigin(origins = "*")
public class VaultController {

    @Autowired
    private VaultService vaultService;

    @Autowired
    private AuditService auditService;

    @PostMapping("/files/add")
    public ResponseEntity<FileResponseDTO> addFile(@RequestBody FileRequestDTO request,
                                                   @RequestHeader("username") String username) {
        FileResponseDTO response = vaultService.addFile(request, username);
        return ResponseEntity.ok(response);
    }

    @GetMapping("/files/all")
    public ResponseEntity<List<FileResponseDTO>> getAllFiles() {
        List<FileResponseDTO> files = vaultService.getAllFiles();
        return ResponseEntity.ok(files);
```

```
    }

    @PostMapping("/files/verify/{id}")
    public ResponseEntity<VerifyResponseDTO> verifyFile(@PathVariable Long id,
                                                        @RequestHeader("username") String username) {
        VerifyResponseDTO response = vaultService.verifyFile(id, username);
        return ResponseEntity.ok(response);
    }

    @GetMapping("/audit/all")
    public ResponseEntity<List<AuditLog>> getAllAuditLogs() {
        List<AuditLog> logs = auditService.getAllAuditLogs();
        return ResponseEntity.ok(logs);
    }
}
```

VaultController.java

Source History

```java
package com.securevault.controller;

import com.securevault.dto.FileRequestDTO;
import com.securevault.dto.FileResponseDTO;
import com.securevault.dto.VerifyResponseDTO;
import com.securevault.model.AuditLog;
import com.securevault.service.AuditService;
import com.securevault.service.VaultService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import java.util.List;

@RestController
@RequestMapping("/api")
@CrossOrigin(origins = "*")
public class VaultController {

    @Autowired
    private VaultService vaultService;

    @Autowired
    private AuditService auditService;

    @PostMapping("/files/add")
    public ResponseEntity<FileResponseDTO> addFile(@RequestBody FileRequestDTO request,
                                                   @RequestHeader("username") String username) {
        FileResponseDTO response = vaultService.addFile(request, username);
        return ResponseEntity.ok(response);
    }

    @GetMapping("/files/all")
    public ResponseEntity<List<FileResponseDTO>> getAllFiles() {
        List<FileResponseDTO> files = vaultService.getAllFiles();
        return ResponseEntity.ok(files);
    }

    @PostMapping("/files/verify/{id}")
    public ResponseEntity<VerifyResponseDTO> verifyFile(@PathVariable Long id,
                                                        @RequestHeader("username") String username) {
        VerifyResponseDTO response = vaultService.verifyFile(id, username);
        return ResponseEntity.ok(response);
    }

    @GetMapping("/audit/all")
    public ResponseEntity<List<AuditLog>> getAllAuditLogs() {
        List<AuditLog> logs = auditService.getAllAuditLogs();
        return ResponseEntity.ok(logs);
    }
}
```

Business Logic

VaultService.java

```java
package com.securevault.service;


import com.securevault.dto.FileRequestDTO;
```

```
import com.securevault.dto.FileResponseDTO;
import com.securevault.dto.VerifyResponseDTO;
import com.securevault.model.FileMetadata;
import com.securevault.repository.FileMetadataRepository;
import com.securevault.util.DateUtil;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import java.util.List;
import java.util.stream.Collectors;

@Service
public class VaultService {

    @Autowired
    private FileMetadataRepository fileMetadataRepository;

    @Autowired
    private HashService hashService;

    @Autowired
    private RiskAnalysisService riskAnalysisService;

    @Autowired
    private AuditService auditService;

    @Transactional
    public FileResponseDTO addFile(FileRequestDTO request, String username) {
        // Generate hash
```

```
    String hash = hashService.generateFileHash(
        request.getFileName(),
        request.getOwner(),
        request.getSize()
    );

    // Analyze risk
    String riskLevel = riskAnalysisService.analyzeRiskLevel(request.getFileName());

    // Create file metadata
    FileMetadata metadata = new FileMetadata(
        request.getFileName(),
        request.getOwner(),
        request.getFileType(),
        request.getSize(),
        DateUtil.getCurrentDateTime(),
        hash,
        riskLevel
    );

    FileMetadata saved = fileMetadataRepository.save(metadata);

    // Log action
    auditService.logAction(username, "Added file: " + request.getFileName() +
        " (Risk: " + riskLevel + ")");

    return convertToDTO(saved);
}
```

```
public List<FileResponseDTO> getAllFiles() {
    return fileMetadataRepository.findAll().stream()
        .map(this::convertToDTO)
        .collect(Collectors.toList());
}

public VerifyResponseDTO verifyFile(Long fileId, String username) {
    FileMetadata file = fileMetadataRepository.findById(fileId)
        .orElseThrow(() -> new RuntimeException("File not found with id: " + fileId));

    // Recalculate hash using the same method
    String currentHash = hashService.generateFileHash(
        file.getFileName(),
        file.getOwner(),
        file.getSize()
    );

    String status = currentHash.equals(file.getHashValue()) ? "VERIFIED" : "TAMPERED";
    String message = status.equals("VERIFIED") ?
        "File integrity is intact" :
        "File has been modified!";

    // Log action
    auditService.logAction(username, "Verified file: " + file.getFileName() +
        " - Status: " + status);

    return new VerifyResponseDTO(
        fileId,
        file.getFileName(),
```

```
            file.getHashValue(),
            currentHash,
            status,
            message
        );
    }

    private FileResponseDTO convertToDTO(FileMetadata metadata) {
        return new FileResponseDTO(
            metadata.getId(),
            metadata.getFileName(),
            metadata.getOwner(),
            metadata.getFileType(),
            metadata.getSize(),
            metadata.getUploadDate(),
            metadata.getHashValue(),
            metadata.getRiskLevel()
        );
    }
}
```

VaultController.java × VaultService.java ×

Source History

```java
    private FileMetadataRepository fileMetadataRepository;

    @Autowired
    private HashService hashService;

    @Autowired
    private RiskAnalysisService riskAnalysisService;

    @Autowired
    private AuditService auditService;

    @Transactional
    public FileResponseDTO addFile(FileRequestDTO request, String username) {
        // Generate hash
        String hash = hashService.generateFileHash(
            request.getFileName(),
            request.getOwner(),
            request.getSize()
        );

        // Analyze risk
        String riskLevel = riskAnalysisService.analyzeRiskLevel(request.getFileName());

        // Create file metadata
        FileMetadata metadata = new FileMetadata(
            request.getFileName(),
            request.getOwner(),
            request.getFileType(),
            request.getSize(),
            DateUtil.getCurrentDateTime(),
            hash,
            riskLevel
        );

        FileMetadata saved = fileMetadataRepository.save(metadata);

        // Log action
        auditService.logAction(username, "Added file: " + request.getFileName() +
            " (Risk: " + riskLevel + ")");

        return convertToDTO(saved);
    }

```

Database Layer (JDBC happens underneath here)

FileMetadataRepository.java

package com.securevault.repository;

import com.securevault.model.FileMetadata;

import org.springframework.data.jpa.repository.JpaRepository;

```
import org.springframework.stereotype.Repository;

import java.util.List;


@Repository

public interface FileMetadataRepository extends JpaRepository<FileMetadata, Long> {

    List<FileMetadata> findByOwner(String owner);

    List<FileMetadata> findByRiskLevel(String riskLevel);

}
```

FileMetadataRepository.java

Source History

```
1   package com.securevault.repository;
2
3   import com.securevault.model.FileMetadata;
4   import org.springframework.data.jpa.repository.JpaRepository;
5   import org.springframework.stereotype.Repository;
6   import java.util.List;
7
8   @Repository
9   public interface FileMetadataRepository extends JpaRepository<FileMetadata, Long> {
10      List<FileMetadata> findByOwner(String owner);
11      List<FileMetadata> findByRiskLevel(String riskLevel);
12  }
```

Database Connection (THIS is JDBC)

application.properties

```
# Server Configuration

server.port=8081


# Database Configuration

spring.datasource.url=jdbc:mysql://localhost:3306/securevault?createDatabaseIfNotExist=true&useSSL=false&allowPublicKeyRetrieval=true&serverTimezone=UTC

spring.datasource.username=root
```

spring.datasource.password=114114

spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver

JPA Configuration

spring.jpa.hibernate.ddl-auto=update

spring.jpa.show-sql=true

spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQL8Dialect

spring.jpa.properties.hibernate.format_sql=true

Logging

logging.level.org.springframework.web=INFO

logging.level.com.securevault=DEBUG

Search Results × FileMetadataRepository.java × application.properties ×

Source History

```
# Server Configuration
server.port=8081

# Database Configuration
spring.datasource.url=jdbc:mysql://localhost:3306/securevault?createDatabaseIf
spring.datasource.username=root
spring.datasource.password=114114
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver

# JPA Configuration
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQL8Dialect
spring.jpa.properties.hibernate.format_sql=true

# Logging
logging.level.org.springframework.web=INFO
logging.level.com.securevault=DEBUG
```

Database Entity (table mapping)

Output:

http://localhost:8081/api/files/all

Pretty print


```
[
  {
    "id": 1,
    "fileName": "finance",
    "owner": "mk",
    "fileType": "pdf",
    "size": 1024,
    "uploadDate": "2026-03-01T15:10:52.459696",
    "hashValue": "ed8741c19facad5a74d4ab85bbb6f9a00bbc0954a72e9861fa51f28c18ccfa5c",
    "riskLevel": "LOW"
  }
]
```

Output

JavaApplication22 (clean) | Run (SecureVaultApplication)

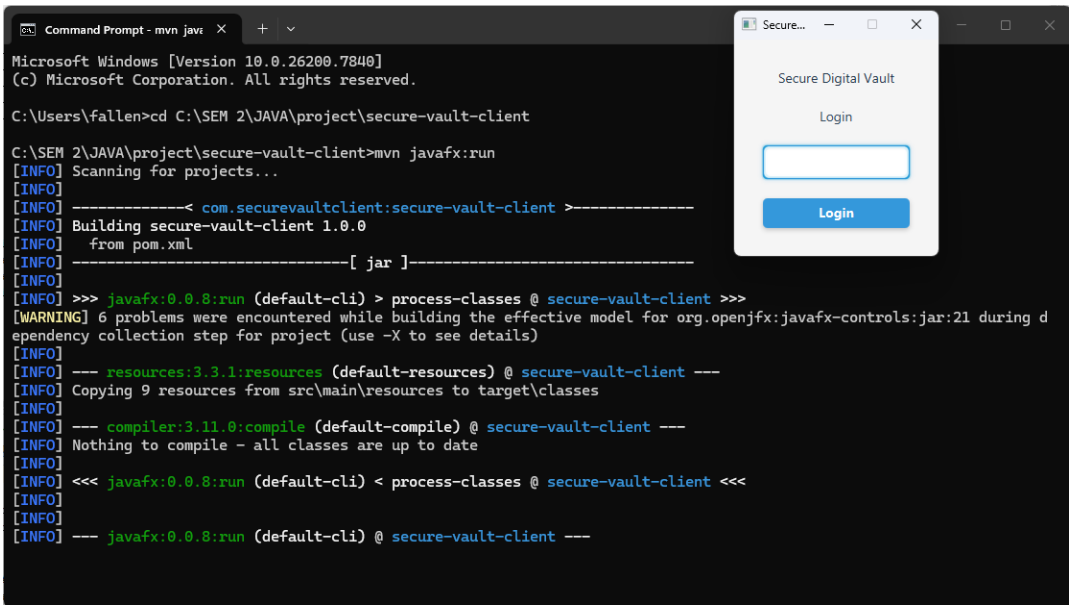
```
----------------< com.securevault:secure-vault-server >-----------------
Building secure-vault-server 1.0.0
  from pom.xml
--------------------------------[ jar ]---------------------------------

--- resources:3.3.1:resources (default-resources) @ secure-vault-server ---
Copying 1 resource from src\main\resources to target\classes
Copying 3 resources from src\main\resources to target\classes

--- compiler:3.11.0:compile (default-compile) @ secure-vault-server ---
Nothing to compile - all classes are up to date

--- exec:3.5.1:exec (default-cli) @ secure-vault-server ---

  .   ____          _            __ _ _
 /\\ / ___'_ __ _ _(_)_ __  __ _ \ \ \ \
( ( )\___ | '_ | '_| | '_ \/ _` | \ \ \ \
 \\/  ___)| |_)| | | | | || (_| |  ) ) ) )
  '  |____| .__|_| |_|_| |_\__, | / / / /
 =========|_|==============|___/=/_/_/_/
 :: Spring Boot ::                (v3.1.5)

2026-03-01T23:39:06.302+05:30  INFO 24440 --- [           main] com.securevault.SecureVaultApplic
2026-03-01T23:39:06.304+05:30 DEBUG 24440 --- [           main] com.securevault.SecureVaultApplic
2026-03-01T23:39:06.305+05:30  INFO 24440 --- [           main] com.securevault.SecureVaultApplic
2026-03-01T23:39:07.059+05:30  INFO 24440 --- [           main] .s.d.r.c.RepositoryConfigurationD
2026-03-01T23:39:07.152+05:30  INFO 24440 --- [           main] .s.d.r.c.RepositoryConfigurationD
WARNING: A restricted method in java.lang.System has been called
WARNING: java.lang.System::load has been called by org.apache.tomcat.jni.Library in an unnamed mo
WARNING: Use --enable-native-access=ALL-UNNAMED to avoid a warning for callers in this module
WARNING: Restricted methods will be blocked in a future release unless native access is enabled

2026-03-01T23:39:07.876+05:30  INFO 24440 --- [           main] o.s.b.w.embedded.tomcat.TomcatWeb
2026-03-01T23:39:07.890+05:30  INFO 24440 --- [           main] o.apache.catalina.core.StandardSe
2026-03-01T23:39:07.891+05:30  INFO 24440 --- [           main] o.apache.catalina.core.StandardEn
2026-03-01T23:39:08.006+05:30  INFO 24440 --- [           main] o.a.c.c.C.[Tomcat].[localhost].[/
2026-03-01T23:39:08.006+05:30  INFO 24440 --- [           main] w.s.c.ServletWebServerApplication
```

Search Results | desktop.ini | SecureVaultApplication.java

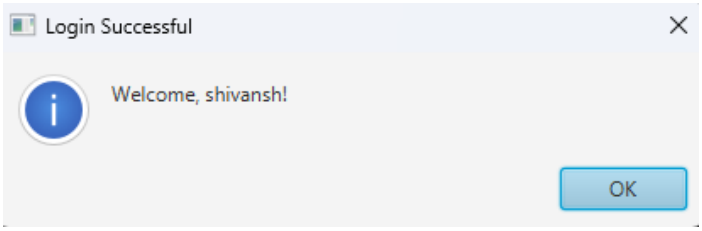
Source | History

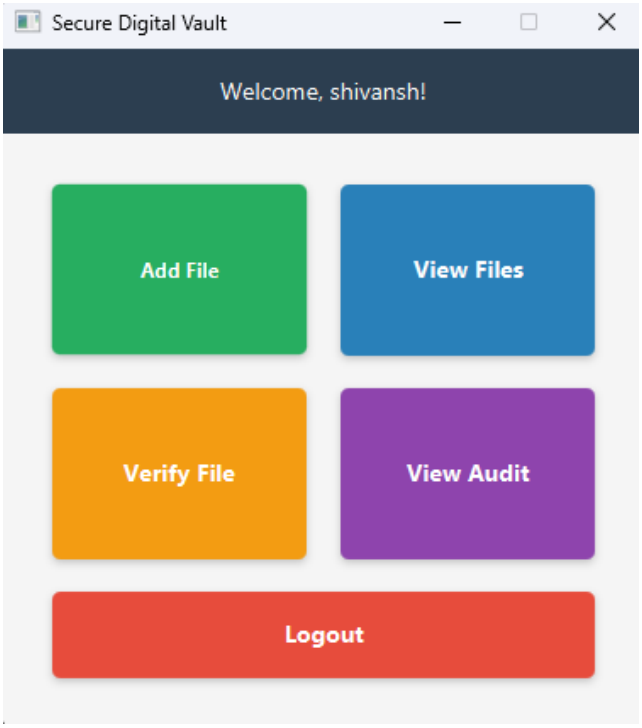
```java
package com.securevault;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class SecureVaultApplication {
    public static void main(String[] args) {
        SpringApplication.run(SecureVaultApplication.class, args);
    }
}
```

JavaFX Login window

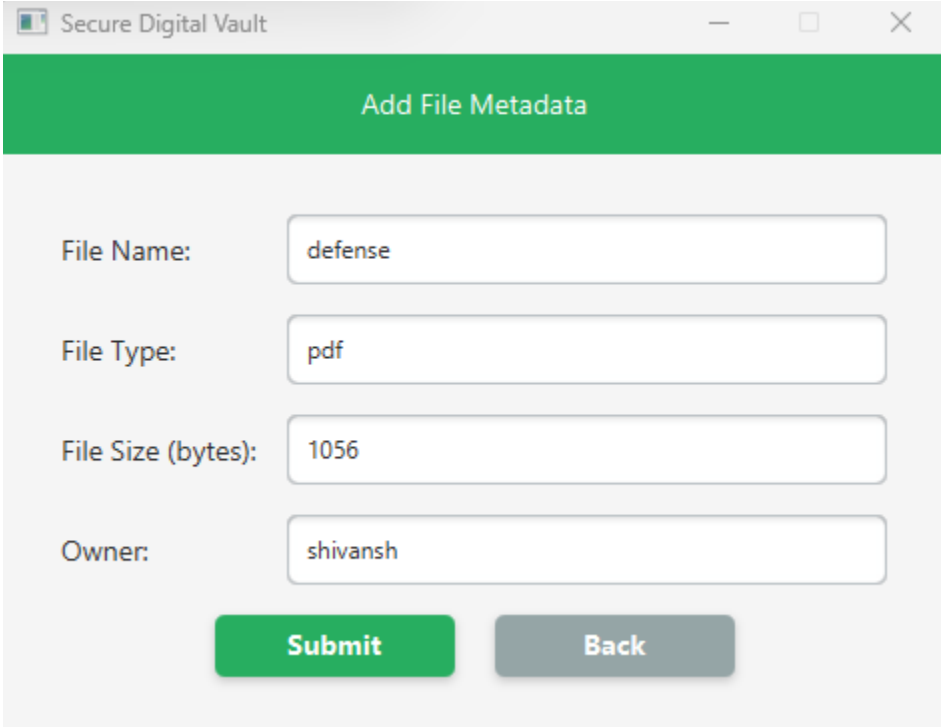


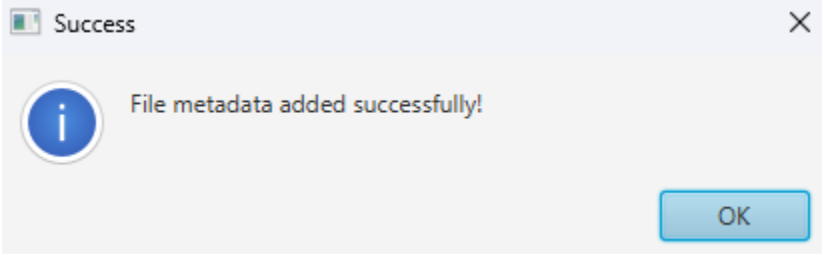
Dashboard window



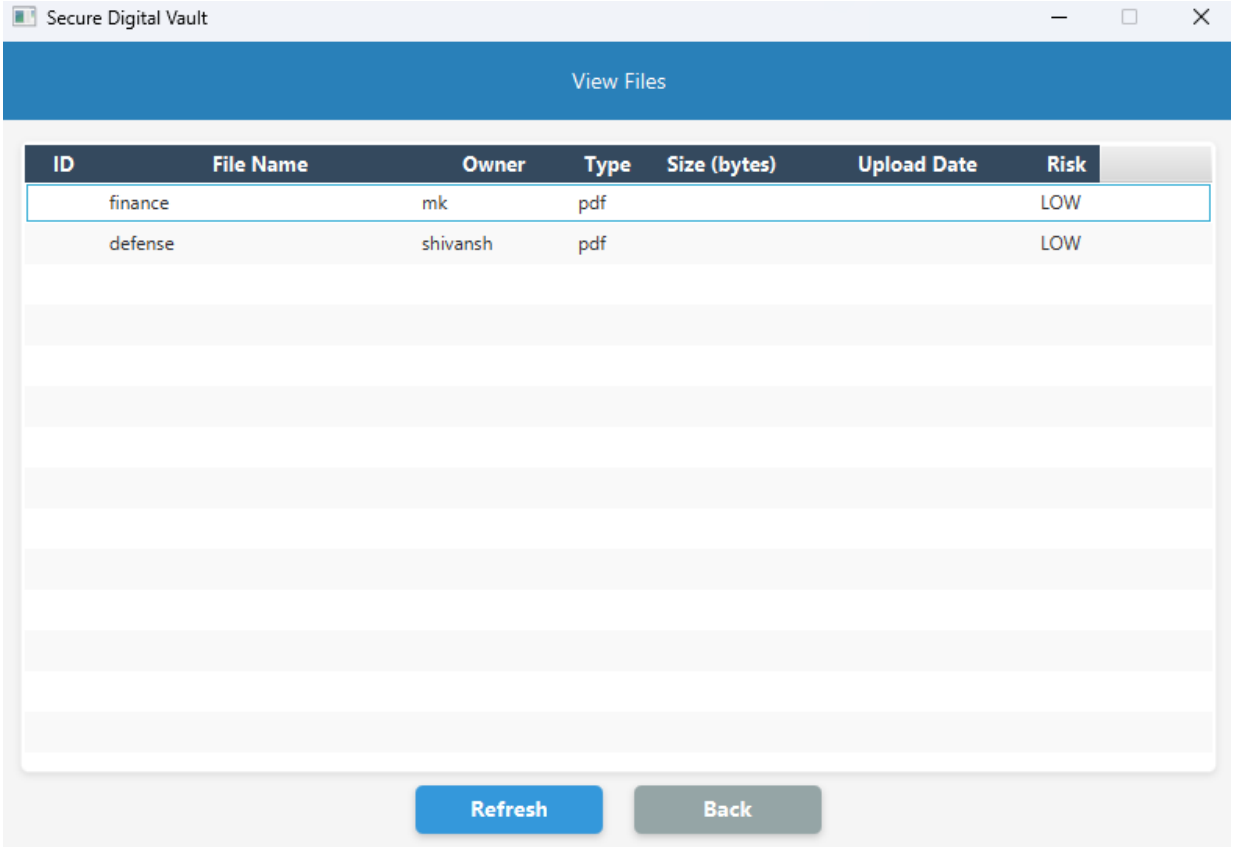
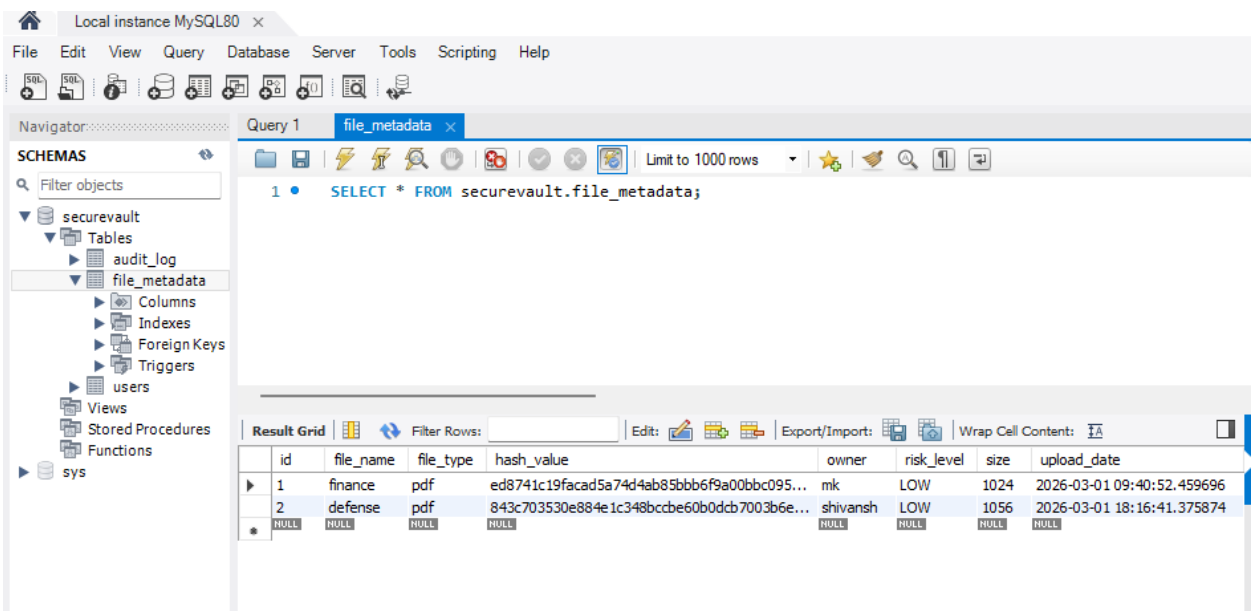


Add file form:





files table



MySQL table showing inserted record (Workbench)